THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

PROJECT REPORT

VibeLearn: AI-Powered Personalized Learning and Adaptive Revision Platform

Submitted by

Abhijay Nagal

B.E. Computer Science and Engineering
4th Semester | Academic Year 2025–2026

Project Type: Independent Personal Software Engineering Project
Current Version: Phase-I MVP

June 2026

Declaration

I, Abhijay Nagal, a student of the Department of Computer Science and Engineering at Thapar Institute of Engineering and Technology, Patiala, hereby declare that this project report titled "VibeLearn: AI-Powered Personalized Learning and Adaptive Revision Platform" has been independently prepared by me as part of a personal software engineering initiative undertaken during the summer recess following my fourth semester of study.

I further declare that:

- This project and the accompanying report represent my original work and have not been submitted, either in part or in full, to any other institution or examination body for the award of any degree, diploma, or certificate.
- All external libraries, frameworks, APIs, and tools used in the course of this project have been duly acknowledged in the References and Appendix sections of this report.
- The architectural decisions, implementation strategies, and engineering tradeoffs documented herein reflect my own analysis and understanding of the problem domain.
- Any assumptions made in the absence of precise empirical data have been explicitly identified and labelled as engineering assumptions within the respective sections.
- The codebase, database schema, and API design described in this report were entirely conceived, designed, and implemented by me, with no academic dishonesty of any form.

Abhijay Nagal

B.E. Computer Science and Engineering

Thapar Institute of Engineering and Technology

Date: June 2026

Certificate of Completion

This is to certify that the project titled "VibeLearn: AI-Powered Personalized Learning and Adaptive Revision Platform" is a bona fide record of work carried out independently by Abhijay Nagal as a personal software engineering project. The project demonstrates the application of modern full-stack web development principles, artificial intelligence integration, and database design skills acquired during the author's academic coursework in the Department of Computer Science and Engineering at Thapar Institute of Engineering and Technology.

The project was conceived, designed, implemented, tested, and documented entirely by the author without formal supervision. The report reflects the author's understanding of software engineering methodology and best practices, and is suitable for inclusion in a professional portfolio.

Abhijay Nagal

Student, Department of Computer Science and Engineering

Thapar Institute of Engineering and Technology, Patiala

Acknowledgement

The successful completion of this project would not have been possible without the resources, tools, and knowledge accumulated through dedicated academic study and independent exploration. I wish to express my sincere gratitude to the following:

First and foremost, I extend my appreciation to the faculty of the Department of Computer Science and Engineering at Thapar Institute of Engineering and Technology for instilling a rigorous understanding of data structures, algorithms, database management systems, and software engineering principles; all of which formed the intellectual foundation for this project.

I am grateful to the developers and maintainers of the open-source tools and frameworks that made this project feasible: the Next.js team at Vercel for building an exceptional full-stack React framework; the Supabase team for providing an accessible, developer-friendly PostgreSQL-backed platform; and the Groq team for offering a high-throughput LLM (Large Language Model) inference API that made real-time AI feature integration practical. The youtube-transcript package deserves special acknowledgement for abstracting the complex process of caption retrieval into a simple programmatic interface.

I also acknowledge the broader educational content creator community on YouTube whose freely available educational videos provide the raw material that the VibeLearn platform transforms into structured learning experiences.

Abstract

The proliferation of educational video content on platforms such as YouTube has democratised access to high-quality learning material across virtually every discipline. However, passive video consumption remains an inefficient mode of learning: learners frequently lack structured notes, receive no formative assessment, and have no mechanism for identifying and remediating their individual conceptual weaknesses. VibeLearn addresses these limitations by transforming passive video consumption into an active, structured, and continuously adaptive learning experience.

VibeLearn is a full-stack, AI-powered personalised learning platform built upon the Next.js App Router framework, a Supabase-managed PostgreSQL database, and the Groq inference API powered by the Llama 3.3 70B Versatile language model. The platform enables users to create study sessions, attach YouTube educational videos, automatically extract video transcripts, and generate structured Markdown study notes and multiple-choice quizzes through a carefully engineered prompt pipeline. User quiz performance is persistently tracked and analysed to identify weak topics. An adaptive revision engine aggregates these weaknesses alongside session-wide notes to generate revision quizzes focused on previously identified weak topics, creating a basic performance-driven revision cycle.

Previously processed videos and generated notes are retained within the platform, enabling learners to revisit historical study material and continue learning from past sessions.

The Phase-I MVP (Minimum Viable Product) of VibeLearn comprises five application pages, three API routes, five normalised database tables, and eight distinct functional modules, representing approximately 1,500 lines of TypeScript code. The architecture is deliberately lean; a monolithic Next.js application with server-side API routes; to minimise operational complexity while maximising feature completeness within the project timeline. Custom frictionless authentication is implemented via localStorage and React Context, consciously trading security robustness for onboarding velocity in this prototype phase.

This report provides a comprehensive account of the system architecture, database design, AI integration strategy, prompt engineering approach, implementation decisions, engineering tradeoffs, and future development roadmap for VibeLearn. It is written from the perspective of the sole developer following the completion of the Phase-I MVP and is intended to serve as both technical documentation and a reflective account of the engineering process.

Keywords: Adaptive Learning, AI-Powered Education, Next.js, Supabase, Groq API, Llama 3.3, Prompt Engineering, MCQ Generation, Weak Topic Detection, Personalised Revision, Full-Stack Development.

Table of Contents

List of Figures

List of Tables

Chapter 1 – Introduction

1.1 Project Overview

VibeLearn is an AI-powered personalised learning and adaptive revision platform designed to bridge the gap between passive consumption of educational video content and active, structured, and measurable learning. The platform targets learners who rely on YouTube as a primary educational resource and wish to convert those viewing sessions into productive, trackable study experiences.

At its core, VibeLearn orchestrates a four-stage learning loop: transcript extraction, AI-powered note generation, quiz-based formative assessment, and adaptive revision driven by detected weaknesses. Each stage feeds into the next, creating a system where the learning material served to a user becomes progressively more tailored to that user's specific gaps over time.

1.2 Motivation

The author, a second-year Computer Science and Engineering student at Thapar Institute of Engineering and Technology, developed VibeLearn as an independent summer project with twin motivations: to solve a genuine personal learning inefficiency experienced during academic study, and to gain hands-on experience with a modern full-stack technology stack that includes server-side rendering, API design, relational database management, and large language model integration.

The immediate problem is simple: watching a lecture video is passive. Without note-taking, reflection, and self-testing, retention of information degrades rapidly. Most learners lack the discipline or tooling to convert videos into structured notes manually, and even fewer engage in systematic identification and remediation of their weak topics. VibeLearn automates these activities.

1.3 Report Organisation

This report is organised into the following major sections: Background and Motivation, Problem Statement, Literature Review, System Architecture, Design and Implementation chapters covering each subsystem in depth, Testing, Results, Limitations, Future Scope, and Conclusion. Diagrams, code excerpts, and tabular summaries are embedded throughout to aid comprehension.

Chapter 2 – Background and Motivation

2.1 The YouTube-as-Education Phenomenon

YouTube has evolved far beyond its origins as an entertainment platform. Channels dedicated to programming tutorials, mathematics, physics, history, and virtually every academic discipline now collectively host billions of instructional minutes of content. Studies in educational technology have repeatedly demonstrated that self-directed learners increasingly supplement or replace formal instruction with online video content. However, the pedagogical literature is equally clear on one point: watching is not learning. The act of passive observation without active processing; note-taking, self-testing, spaced repetition; produces shallow encoding and poor long-term retention.

2.2 The Forgetting Curve and Active Learning

Hermann Ebbinghaus's seminal work on memory and forgetting established that without deliberate review, humans forget approximately 50% of newly learned information within an hour, and nearly 70% within 24 hours. The remedy, confirmed by decades of subsequent research, is active retrieval practice; the act of testing oneself on material; which has been shown to dramatically improve retention compared to passive re-reading or re-watching. VibeLearn's quiz engine and adaptive revision system are directly informed by this principle.

2.3 Limitations of Existing Solutions

While tools such as Coursera, Udemy, Khan Academy, and Notion-based note systems exist, none provides the specific combination of: arbitrary YouTube video compatibility, fully automated AI-generated notes, MCQ quiz generation tied to specific video content, weak-topic detection, and adaptive revision that adjusts to individual performance. This gap defines the precise niche that VibeLearn occupies.

2.4 Developer's Learning Goals

Beyond solving the educational problem, this project served as a deliberate technical learning exercise. Specifically, the author sought hands-on experience with the Next.js App Router (released as stable in Next.js 13 and refined through 14), Supabase as a Backend-as-a-Service (BaaS) platform, TypeScript-first development discipline, REST API design within a Next.js monolith, and prompt engineering for structured output generation using large language models. Each of these technologies is directly relevant to modern Software Development Engineer (SDE) roles, which represent the author's primary career objective.

Chapter 3 – Problem Statement

3.1 Core Problem

Students and self-directed learners who use YouTube as an educational resource face three interconnected problems that collectively undermine learning effectiveness:

1. Absence of structured notes: Video content is inherently temporal and non-persistent. Unlike textbooks, videos do not leave a tangible artefact behind. Learners who watch without note-taking are left with only vague memories.
2. Lack of formative assessment: Without quizzes or self-testing mechanisms tied directly to the content they have just consumed, learners cannot gauge their comprehension or identify gaps.
3. No adaptive remediation: Even learners who do quiz themselves typically do so uniformly, without systematic identification of their specific weak areas and targeted generation of revision material focused on those areas.

3.2 Problem Scope

The problem is scoped to English-language YouTube educational videos for which auto-generated or manually created captions are available. The platform assumes learners have basic web literacy and access to a browser. It does not address video content without available transcripts, nor does it attempt to replace formal educational instruction.

3.3 Desired Outcome

A successful solution would: automatically extract structured, readable notes from any YouTube educational video; generate contextually accurate multiple-choice questions tied to that content; track user performance over time; and generate revision material that disproportionately targets identified weak areas; all within a seamless, low-friction user interface.

Chapter 4 – Objectives

4.1 Primary Objectives

- Design and implement a full-stack web application that enables session-based YouTube learning workflows.
- Integrate an LLM (Large Language Model) API to generate structured, topic-organised Markdown notes from video transcripts.
- Develop an MCQ (Multiple Choice Question) quiz engine that generates, administers, and evaluates quizzes derived from video content.
- Implement persistent performance tracking that records quiz scores and identifies weak topics at a granular level.
- Build an adaptive revision engine that aggregates session-level weaknesses and generates personalised revision quizzes.

4.2 Secondary Objectives

- Design a clean, normalised relational database schema in PostgreSQL via Supabase.
- Implement a friction-free authentication system suitable for a prototype application.
- Produce a maintainable, TypeScript-first codebase of approximately 1,500 lines.
- Deliver a functional MVP within a single development sprint.
- Document the full project to professional software engineering report standards.

Chapter 5 – Scope of the Project

5.1 In Scope

- YouTube URL ingestion and video ID extraction.
- Transcript extraction using the youtube-transcript npm package.
- AI note generation using the Groq API (Llama 3.3 70B Versatile model).
- AI MCQ quiz generation per video.
- Persistent storage of users, sessions, videos, quizzes, and quiz attempts in Supabase PostgreSQL.
- Session-based study workflow management.
- Adaptive revision quiz generation based on aggregated weak topics.
- Custom username-based authentication using localStorage and React Context.
- Responsive user interface built with Next.js, Tailwind CSS, and Shadcn/UI.

5.2 Out of Scope (Phase-I)

- OAuth or JWT (JSON Web Token) based authentication.
- Support for non-YouTube video sources (e.g., Vimeo, local uploads).
- Multi-language transcript support.
- Spaced repetition scheduling algorithms (e.g., SM-2).
- Mobile-native application.
- Real-time collaboration or social learning features.
- Video content without available captions.
- Payment integration or subscription management.

Chapter 6 – Literature Review

6.1 Adaptive Learning Systems

Adaptive learning systems, also referred to as Intelligent Tutoring Systems (ITS) in the educational technology literature, have been studied since at least the 1970s. Seminal work by Sleeman and Brown (1982) established the foundational framework of model-tracing tutors that maintain an explicit model of student knowledge and adapt instruction accordingly. More recent systems such as Carnegie Learning's MATHia use Bayesian Knowledge Tracing (BKT) to estimate the probability that a student has mastered a skill, and adjust the difficulty and focus of practice problems accordingly.

VibeLearn implements a simplified variant of this adaptive principle. Rather than maintaining a probabilistic knowledge model, it tracks weak topics as explicitly identified string labels extracted from quiz performance data and uses these labels as conditioning context for subsequent revision quiz generation. This approach trades mathematical rigour for implementation simplicity; an appropriate tradeoff for a Phase-I MVP.

6.2 Automated Question Generation

The automated generation of assessment questions from educational text is an active research domain. Classical approaches used natural language processing (NLP) techniques such as named entity recognition, dependency parsing, and sentence transformation rules to produce fill-in-the-blank or WH-type questions. The emergence of transformer-based LLMs; particularly GPT-3 (Brown et al., 2020) and its successors; has dramatically simplified this task: a well-crafted prompt can instruct a large language model to produce pedagogically appropriate MCQs with distractors and explanations in a single inference call.

The quality of LLM-generated questions depends critically on the quality of the source context provided and the specificity of the prompt. Research by Elkins et al. (2023) found that GPT-4-generated MCQs were rated as comparable in quality to instructor-authored questions in a blind evaluation study. VibeLearn leverages this capability using the Llama 3.3 70B model via Groq, with carefully structured prompts that specify question format, JSON output schema, and the requirement for explanations accompanying each answer.

6.3 Transcript Utilisation in EdTech

The use of video transcripts as a source of educational content is well-established. Platforms such as Coursera and edX expose transcripts alongside videos to support learners with hearing impairments and those who prefer text-based review. Academic work by Bunt et al. (2014) demonstrated that transcript-based search and navigation significantly improved learner efficiency in video-based courses. VibeLearn extends this concept by not merely exposing the transcript but by using it as the basis for AI-generated structured content; a transformation from raw, verbose transcript text into concise, topic-organised study notes.

6.4 Existing Platform Analysis

Platform	Auto Notes	MCQ Gen	Adaptive	YouTube	Free
Coursera	No	Partial	Partial	No	Partial
Khan Academy	No	Yes	Yes	No	Yes
Notion AI	Manual	No	No	No	Partial
Quizlet	No	Partial	No	No	Partial
VibeLearn	Yes (AI)	Yes (AI)	Yes	Yes	Yes

Table 1: Comparison of Existing Learning Platforms

Chapter 7 – Proposed System

7.1 System Concept

VibeLearn proposes a closed-loop learning system that functions across two distinct phases: the Learning Phase and the Adaptive Revision Phase. In the Learning Phase, users create study sessions, add YouTube videos, receive AI-generated notes and quizzes, and have their performance recorded. In the Adaptive Revision Phase, the system analyses accumulated performance data to identify weak topics and generates a personalised revision quiz targeting those specific areas.

7.2 Key Differentiators

- Full automation: No manual note-taking or quiz creation required from the user.
- Content grounding: All generated notes and questions are grounded in the actual content of the specified video, not in generic topic knowledge.
- Session-level adaptation: Revision is adaptive not merely at the video level but across all videos in a session, providing a holistic view of learning gaps.
- Continuous loop: Each revision attempt generates fresh performance data which further refines future revisions.

7.3 System Boundaries

The system boundary encompasses the browser client, the Next.js server, the Groq API, and the Supabase database. The YouTube platform is an external dependency accessed only for transcript data; no video hosting or streaming occurs within VibeLearn. The Groq API is similarly an external service; VibeLearn depends on its availability but does not manage model weights or inference infrastructure.

Chapter 8 – Requirement Analysis

8.1 Functional Requirements

ID	Module	Requirement	Priority
FR-01	Auth	System shall allow new users to register with a unique username	High
FR-02	Auth	System shall persist login state across browser sessions	High
FR-03	Session Mgmt	Users shall be able to create named study sessions	High
FR-04	Session Mgmt	Users shall be able to end and archive sessions	High
FR-05	Video	System shall accept a YouTube URL and extract the video ID	High
FR-06	Video	System shall retrieve the transcript of the specified video	High
FR-07	Notes	System shall generate structured Markdown notes from the transcript	High
FR-08	Notes	Notes shall be persistently stored and retrievable at any time	High
FR-09	Quiz	System shall generate MCQ quizzes from video notes	High
FR-10	Quiz	System shall evaluate quiz responses and provide explanations	High
FR-11	Tracking	System shall persist quiz scores and weak topic labels per attempt	High
FR-12	Revision	System shall generate adaptive revision quizzes targeting weak topics	High
FR-13	Dashboard	Dashboard shall show active and completed sessions	Medium
FR-14	UI	YouTube thumbnails shall be displayed alongside video entries	Low

Table 2: Functional Requirements

8.2 Non-Functional Requirements

ID	Category	Requirement	Priority
NFR-01	Performance	Note generation shall complete within 15 seconds for videos up to 60 minutes	High
NFR-02	Performance	Quiz generation shall complete within 10 seconds	High
NFR-03	Usability	Core workflows shall be completable in fewer than 5 user actions	High
NFR-04	Reliability	Database writes shall be transactionally consistent	High
NFR-05	Scalability	Architecture shall support horizontal scaling via stateless API routes	Medium
NFR-06	Maintainability	All modules shall be TypeScript-typed with no implicit any	Medium
NFR-07	Security	API keys shall never be exposed to the browser client	High
NFR-08	Accessibility	UI shall be keyboard-navigable and screen-reader compatible	Low

Table 3: Non-Functional Requirements

Chapter 9 – System Architecture

9.1 High-Level Architecture

VibeLearn follows a monolithic full-stack architecture pattern using Next.js as the unified frontend and backend runtime. This architecture choice is deliberate: for a Phase-I MVP, a monolith minimises infrastructure complexity, eliminates cross-origin request issues, simplifies deployment, and allows rapid feature iteration. The architectural layers are as follows:

Architecture Diagram (Mermaid):

```
graph TD
  A[Browser - React / Next.js App Router] -->|HTTP requests| B[Next.js Server]
  B -->|API Routes /api/*| C[TypeScript Handler Functions]
  C -->|Groq SDK calls| D[Groq API - Llama 3.3 70B]
  C -->|Supabase JS Client| E[Supabase BaaS]
  E -->|SQL| F[PostgreSQL Database]
  C -->|npm package| G[youtube-transcript]
  G -->|HTTP| H[YouTube Caption Servers]
  B -->|SSR / RSC| A
```

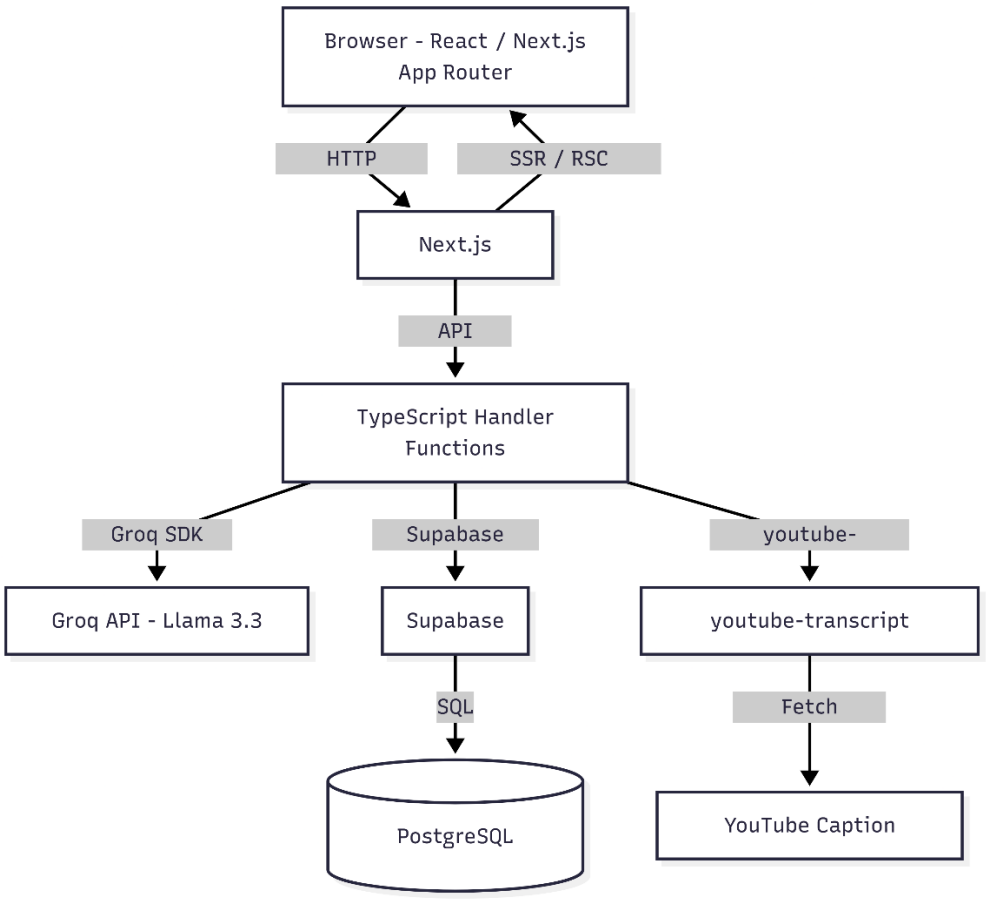

Figure 9: High-Level System Architecture

9.2 Architectural Layers

9.2.1 Presentation Layer

The presentation layer consists of Next.js App Router pages and React components. Pages are rendered using a combination of Server Components (for initial data fetching) and Client Components (for interactive quiz elements, form submissions, and real-time state management). Tailwind CSS provides utility-first styling, while Shadcn/UI components (built on Radix UI primitives) supply accessible, pre-designed UI elements such as buttons, cards, dialogs, and progress indicators. React Markdown is used to render the AI-generated Markdown notes within the browser.

9.2.2 API Layer

The API layer consists of three Next.js Route Handler functions located in the app/api directory. These functions execute exclusively on the server, preventing client-side exposure of sensitive credentials (Groq API key, Supabase service role key). Each route handler orchestrates a specific workflow: note generation, quiz generation, and revision quiz generation. The handlers follow a consistent pattern of input validation, external service invocation, database persistence, and JSON response.

9.2.3 Data Layer

The data layer is managed by Supabase, which provides a PostgreSQL database, a JavaScript client library, and a web-based dashboard for schema management and query inspection. The Supabase JS client is used exclusively within server-side API route handlers. Row-level security is noted as a future enhancement; in Phase-I, table access is managed through the service role key within server functions only.

9.2.4 AI Inference Layer

The AI inference layer is provided by the Groq API, which offers high-throughput inference for open-source models. The Llama 3.3 70B Versatile model is selected for its strong instruction-following capability, JSON output compliance, and superior performance on structured generation tasks. The Groq SDK handles authentication, request serialisation, and response parsing. All AI calls are made server-side, ensuring API key security.

Chapter 10 – Detailed Workflow

10.1 Learning Phase Workflow

Learning Phase Sequence Diagram (Mermaid):

```
sequenceDiagram
  participant U as User (Browser)
  participant S as Next.js Server
  participant G as Groq API
  participant DB as Supabase DB
  participant YT as YouTube
  U->>S: POST /api/generate-notes { yt_url, session_id, user_id }
  S->>S: Extract video_id from URL
  S->>YT: youtube-transcript.fetchTranscript(video_id)
  YT-->>S: Raw caption segments []
  S->>S: Concatenate & clean transcript text
  S->>G: chat.completions.create (notes prompt + transcript)
  G-->>S: Markdown notes string
  S->>DB: INSERT into videos (notes, title, thumbnail_url ...)
  DB-->>S: Inserted row
  S-->>U: { notes, video_id }
  U->>S: POST /api/generate-quiz { video_id, notes }
  S->>G: chat.completions.create (quiz prompt + notes)
  G-->>S: JSON quiz object
  S->>DB: INSERT into quizzes (questions JSONB ...)
  DB-->>S: Inserted row
  S-->>U: { quiz }
  U->>U: User attempts quiz
  U->>DB: INSERT into quiz_attempts (score, weak_topics ...)
```

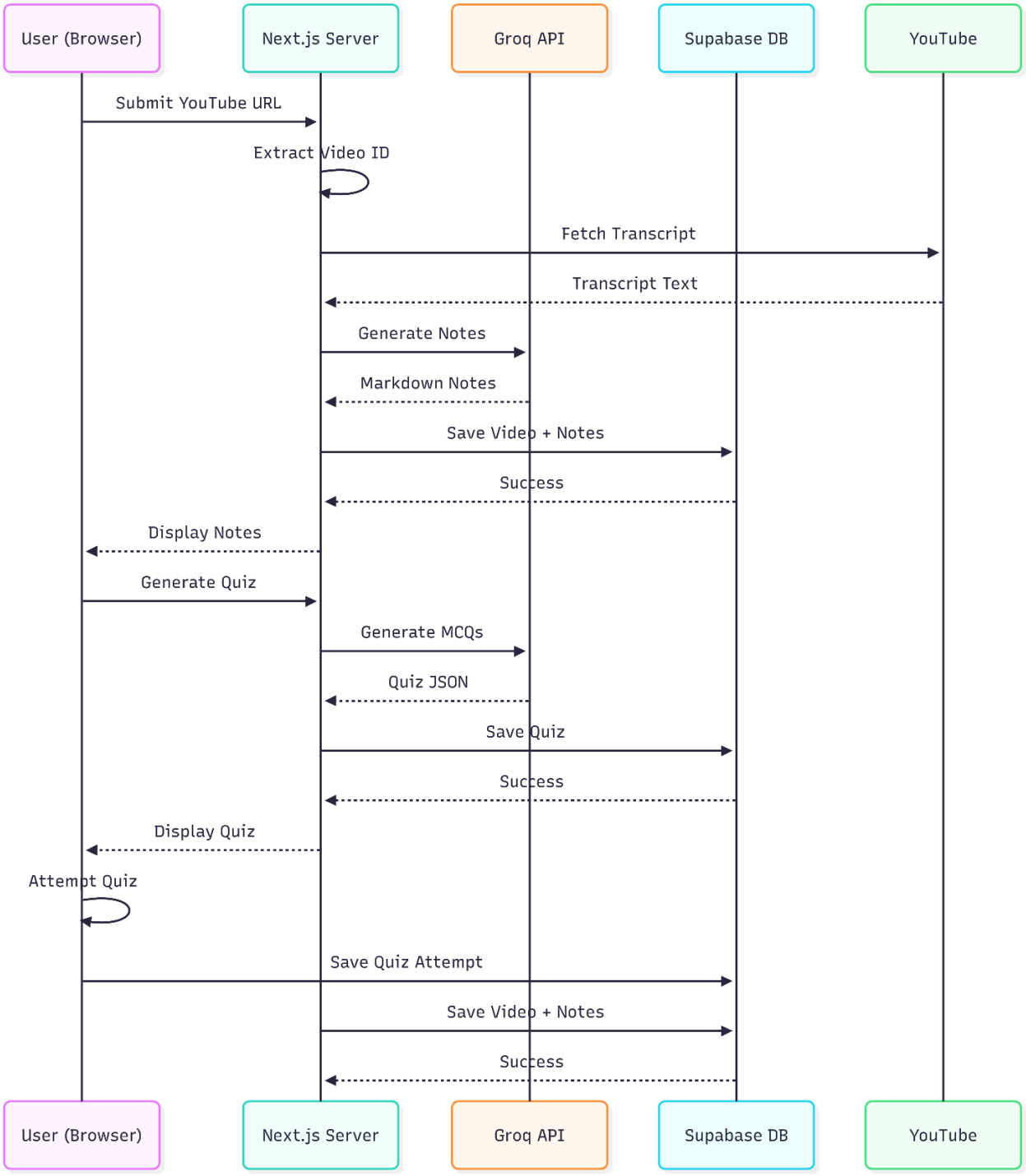

Figure 10: Learning Phase Sequence Diagram

10.2 Adaptive Revision Workflow

Adaptive Revision Flow (Mermaid):

```
flowchart TD
  A[User requests Revision Quiz] --> B[Fetch all quiz_attempts for session]
  B --> C[Extract weak_topics from each attempt]
  C --> D[Aggregate and deduplicate weak topics]
  D --> E[Fetch all notes for all videos in session]
  E --> F[Construct revision prompt with weak topics + notes]
  F --> G[Groq API - Llama 3.3 70B]
  G --> H[Return adaptive MCQ quiz JSON]
  H --> I[Persist as quiz_type=revision_master]
  I --> J[User attempts revision quiz]
  J --> K[Store new quiz_attempt with updated weak_topics]
  K --> L[Loop - next revision is more refined]
```

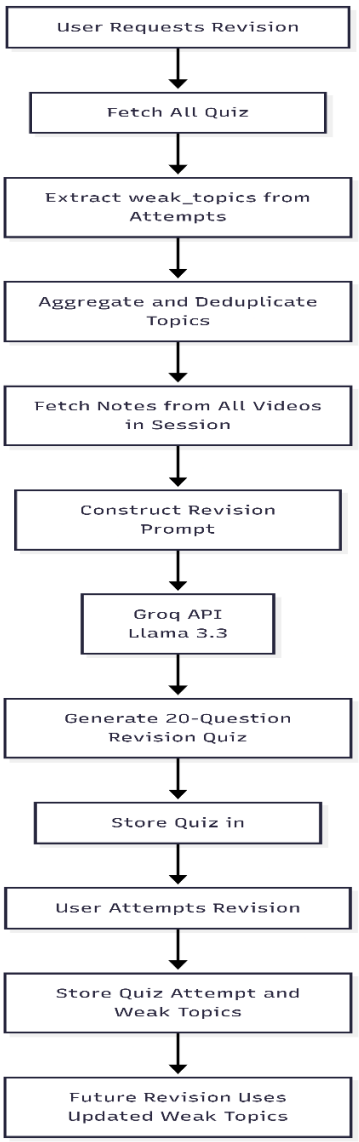

Figure 8: Adaptive Revision Flow Diagram

Chapter 11 – Database Design

11.1 Overview

The VibeLearn database is hosted on Supabase and implemented as a normalised PostgreSQL relational schema comprising five tables. The schema adheres to Third Normal Form (3NF): all non-key attributes are dependent on the primary key, not on other non-key attributes. UUIDs are used as primary keys throughout, generated server-side to avoid sequential key exposure and to support potential future distribution.

11.2 Table Descriptions

Table	Primary Key	Purpose
users	id (UUID)	Stores application users identified by username only; no password stored in Phase-I.
sessions	id (UUID)	Groups videos and quizzes into a named study session; references users.id; status field tracks active vs. completed.
videos	id (UUID)	Stores metadata and AI-generated notes for each processed YouTube video; JSONB notes field stores Markdown string.
quizzes	id (UUID)	Stores generated quizzes; questions stored as JSONB array; quiz_type distinguishes video_quiz from revision_master.
quiz_attempts	id (UUID)	Records each quiz attempt; weak_topics JSONB array stores string labels of topics where user answered incorrectly.

Table 5: Database Table Descriptions

11.3 Schema Details

11.3.1 users Table

```
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  username    TEXT NOT NULL UNIQUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

11.3.2 sessions Table

```
CREATE TABLE sessions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL,
  status      TEXT NOT NULL DEFAULT 'active',  -- 'active' | 'completed'
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

11.3.3 videos Table

```
CREATE TABLE videos (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id    UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  yt_video_id   TEXT NOT NULL,
  yt_url        TEXT NOT NULL,
  title         TEXT,
  thumbnail_url TEXT,
  notes         TEXT,  -- Markdown string
  created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

11.3.4 quizzes Table

```
CREATE TABLE quizzes (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id  UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
  video_id    UUID REFERENCES videos(id) ON DELETE SET NULL,
  quiz_type   TEXT NOT NULL,  -- 'video_quiz' | 'revision_master'
  questions   JSONB NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

11.3.5 quiz_attempts Table

```
CREATE TABLE quiz_attempts (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quiz_id         UUID NOT NULL REFERENCES quizzes(id) ON DELETE CASCADE,
  score           INTEGER NOT NULL,
  total_questions INTEGER NOT NULL,
  weak_topics     JSONB,  -- array of topic label strings
  created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

11.4 Entity Relationship Diagram

ER Diagram (Mermaid):

```
erDiagram
  users ||--o{ sessions : "has many"
  sessions ||--o{ videos : "contains"
  sessions ||--o{ quizzes : "has"
  videos ||--o{ quizzes : "generates"
  quizzes ||--o{ quiz_attempts : "recorded in"
  users { uuid id PK; text username; timestamptz created_at }
  sessions { uuid id PK; uuid user_id FK; text title; text status; timestamptz
created_at }
  videos { uuid id PK; uuid session_id FK; text yt_video_id; text title; text notes;
timestamptz created_at }
  quizzes { uuid id PK; uuid session_id FK; uuid video_id FK; text quiz_type; jsonb
questions; timestamptz created_at }
  quiz_attempts { uuid id PK; uuid quiz_id FK; int score; int total_questions; jsonb
weak_topics; timestamptz created_at }
```

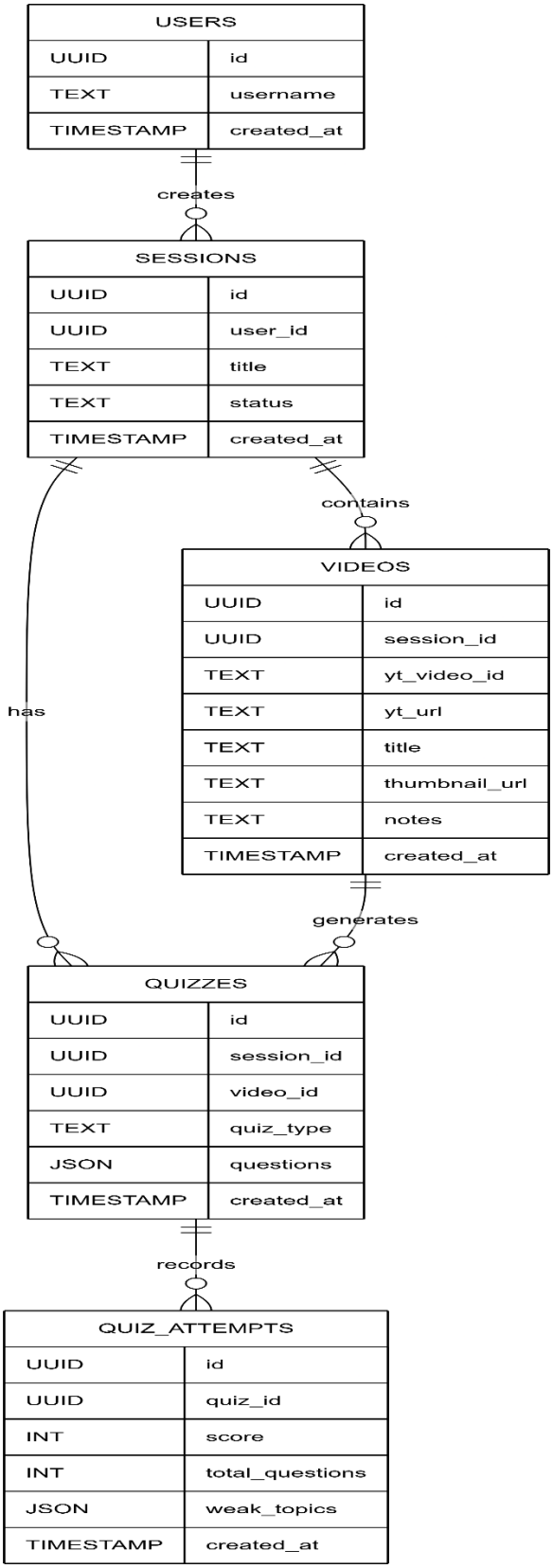

Figure 7: Database Entity-Relationship Diagram

11.5 JSONB Design Decisions

Two columns use PostgreSQL's JSONB type: quizzes.questions and quiz_attempts.weak_topics. The choice of JSONB over normalised relational structures is deliberate. The questions array has a fixed, well-defined schema at the application layer (each question object contains a question string, four option strings, a correct answer index, and an explanation string), but the number of questions per quiz is variable and the internal structure does not need to be independently queried at the database level. JSONB provides the flexibility of a document store for this sub-structure while maintaining the relational integrity of the parent table.

The weak_topics array is similarly variable in length and does not require independent table-level querying. It is read in bulk by the adaptive revision engine and processed entirely in application code. JSONB is therefore the appropriate storage choice. An engineering assumption is made here: in a production system with significantly larger data volumes, normalising weak_topics into a separate table would be advisable to enable analytics queries.

Chapter 12 – Technology Stack Analysis

12.1 Technology Selection Rationale

Technology	Category	Rationale for Selection	Alternatives Considered
Next.js 14	Full-Stack Framework	Unified frontend/backend, App Router, SSR, API Routes, Vercel deployment	Remix, SvelteKit, Express + React
TypeScript	Language	Type safety, IDE tooling, reduced runtime errors, industry standard	JavaScript (ES6+)
Tailwind CSS	Styling	Utility-first, rapid prototyping, no CSS file management overhead	CSS Modules, Styled Components, MUI
Shadcn/UI	Component Library	Accessible, copy-paste components, Radix UI primitives, Tailwind compatible	MUI, Chakra UI, Mantine
Supabase	BaaS / Database	PostgreSQL backbone, JS client library, dashboard, free tier, instant setup	Firebase, PlanetScale, Neon
Groq API	LLM Inference	High throughput, low latency inference, generous free tier, Llama 3.3 70B availability	OpenAI API, Anthropic API, Ollama
Llama 3.3 70B	Language Model	Strong instruction following, JSON compliance, open-source weights, no per-token cost on Groq free tier	GPT-4o, Claude 3 Haiku, Mistral
youtube-transcript	Data Extraction	Simple API, handles both auto-generated and manual captions, actively maintained	Custom yt-dlp wrapper, YouTube Data API v3
React Markdown	Rendering	Safe Markdown-to-HTML rendering with plugin support, lightweight	Marked.js, MDX

Table 4: Technology Stack Summary

Chapter 13 – Frontend Implementation

13.1 Page Structure

The VibeLearn frontend comprises five distinct pages within the Next.js App Router structure. The App Router uses filesystem-based routing where each page.tsx file in a directory becomes a route. The application routes are as follows:

- / (Root): Home page and user onboarding. Users who have not previously logged in are presented with a username creation form. Returning users are redirected to the dashboard.
- /dashboard: The central hub showing active and completed sessions. Allows session creation and navigation.
- /session/[id]: The session workspace. Dynamically routed by session ID. Displays the video addition form, the list of processed videos with their notes, and navigation to quizzes and revision.
- /quiz/[id]: The quiz interface. Renders questions, handles answer selection, evaluates responses, displays explanations, and calculates scores.
- /revision/[id]: The revision interface. Similar to the quiz page but serves revision_master type quizzes.

Figure 1: Home Page / User Onboarding Screen

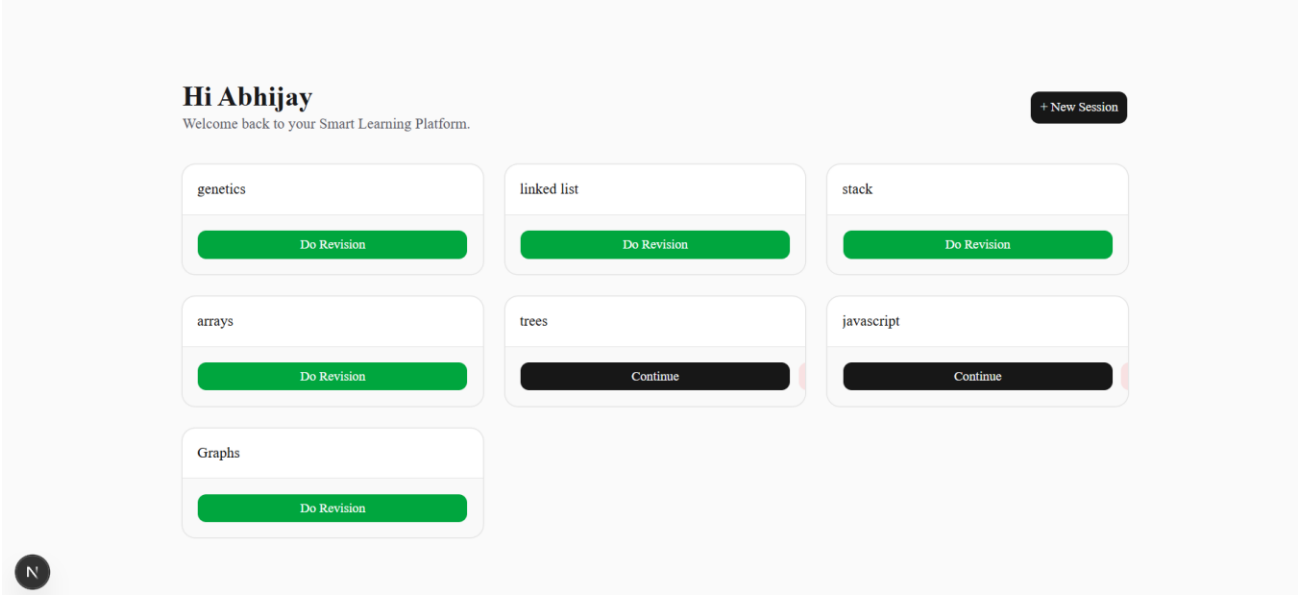

Figure 2: Dashboard – Active and Completed Sessions

13.2 Authentication State Management

Authentication state is managed entirely within the React client. Upon username submission on the home page, the application queries the Supabase users table to check for an existing user with that username. If found, the user's UUID is retrieved. If not found, a new user record is created and the UUID is returned. In both cases, the UUID and username are persisted to localStorage under predefined keys.

A React Context provider (AuthContext) wraps the application at the root layout level. On mount, the AuthContext reads from localStorage and populates the context state. All pages and components that require user identity consume this context via the useAuth hook. This approach ensures a single source of truth for authentication state without any server-side session management.

13.3 Component Architecture

The component hierarchy follows the React composition pattern. Shared components are housed in a /components directory. Key components include: VideoCard (displays a processed video with thumbnail and notes), QuizQuestion (renders a single MCQ with options), ScoreDisplay (summarises quiz performance), SessionCard (represents a session on the dashboard), and a MarkdownRenderer wrapper around React Markdown with custom prose styling applied via Tailwind.

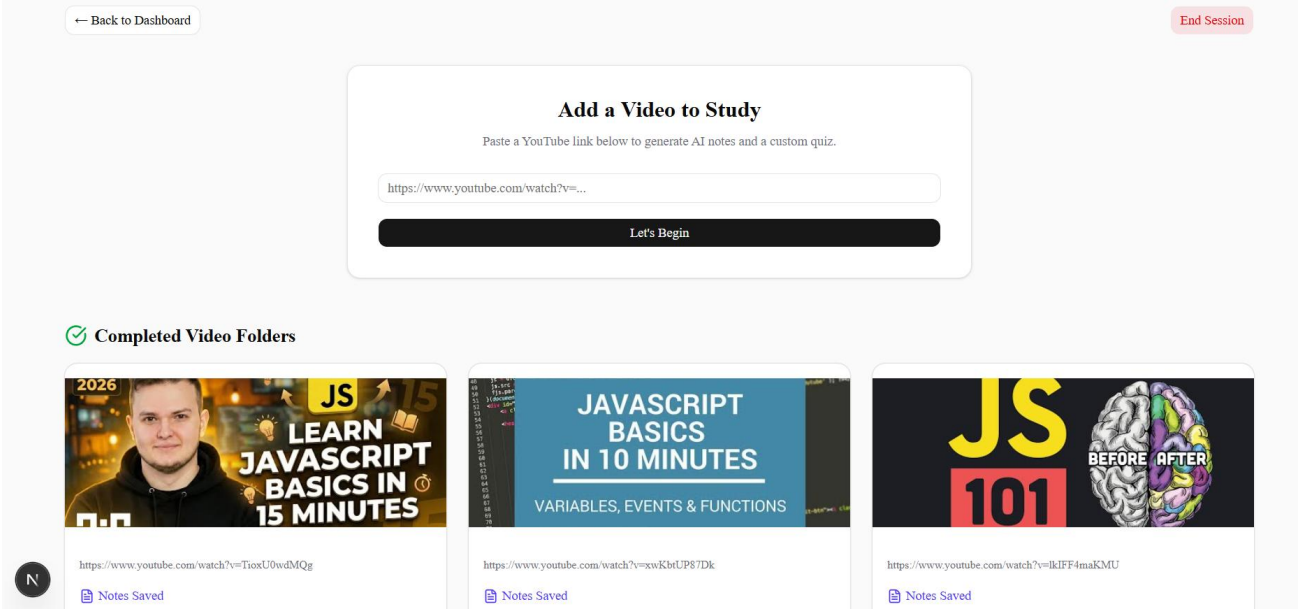

Figure 3: Session Workspace – Video Addition and Notes Panel

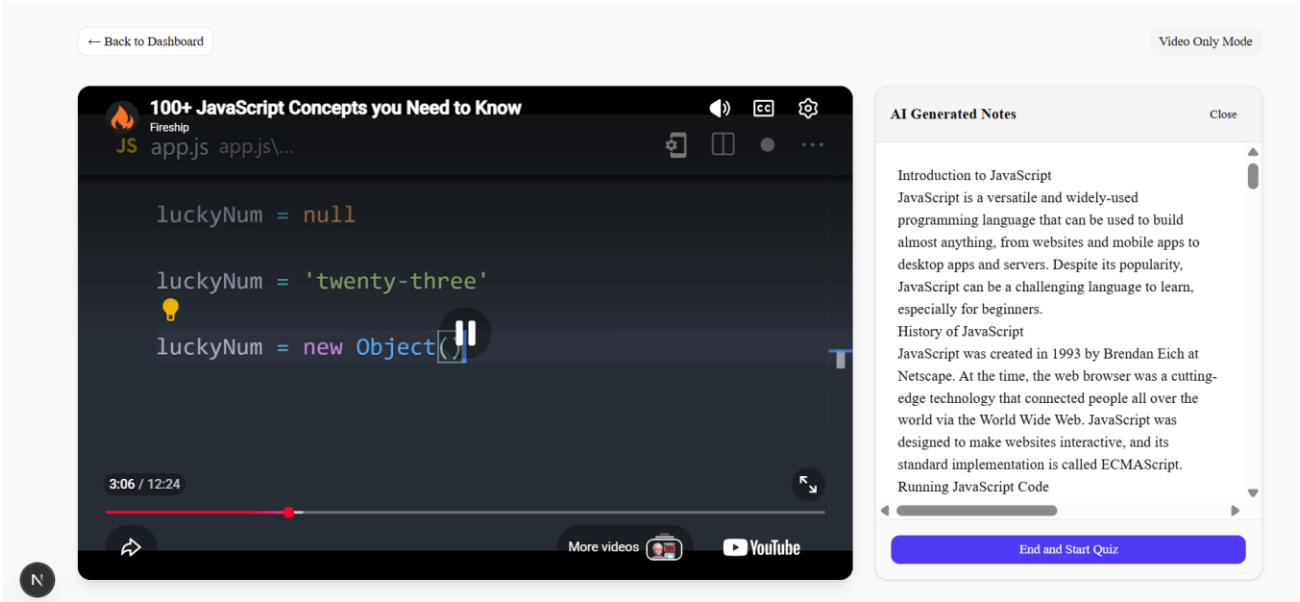

Figure 4: Notes Interface – AI-Generated Markdown Study Notes

13.4 State Management Strategy

Local component state (useState, useReducer) is used for transient UI state such as the current selected quiz answer, loading indicators, and form input values. Server state; data fetched from Supabase; is loaded via async functions within Server Components where possible, and via useEffect in Client Components where interactivity demands it. No global state management library (Redux, Zustand) is used; the application's complexity does not warrant it at this stage.

Chapter 14 – Backend Implementation and API Design

14.1 API Route Architecture

All server-side logic is encapsulated in Next.js Route Handlers under the app/api directory. These handlers are stateless HTTP endpoint functions that execute in the Node.js runtime on the server. Their stateless nature means each request carries all necessary context in its body; no server-side session state is maintained. This design is inherently scalable and compatible with serverless deployment environments such as Vercel.

Route	Method	Input	Output
/api/generate-notes	POST	{ yt_url, session_id, user_id }	{ notes, video_id, title, thumbnail_url }
/api/generate-quiz	POST	{ video_id, notes, session_id }	{ quiz_id, questions[] }
/api/generate-master-quiz	POST	{ session_id }	{ quiz_id, questions[] }

Table 6: API Route Specifications

14.2 /api/generate-notes Implementation

This route handler is the most complex in the application. It orchestrates a five-step pipeline: URL parsing, transcript retrieval, AI inference, metadata enrichment, and database persistence. Upon receiving the request, the handler first validates that a yt_url is present and parses the YouTube video ID using a regular expression that handles both youtu.be short URLs and full youtube.com URLs with query parameters. The youtube-transcript package is then invoked with the extracted video ID.

The raw transcript response is an array of segment objects, each containing a text string and a timestamp offset. These segments are concatenated into a single continuous string, with newlines stripped to reduce token consumption. The concatenated transcript; which for a one-hour video may be 8,000–15,000 words; is then inserted into a carefully structured prompt sent to the Groq API.

The Groq response is a Markdown string, which is stored directly in the videos.notes column. In parallel (engineering assumption: in the current implementation, sequentially), the handler attempts to retrieve the video title and thumbnail URL from the YouTube oEmbed endpoint, a public API that requires no authentication. The video record is then inserted into Supabase and the response is returned to the client.

14.3 /api/generate-quiz Implementation

The quiz generation route receives the video_id and the notes string. The notes; rather than the raw transcript; are used as the context for quiz generation, for two reasons: the notes are significantly shorter than the transcript (reducing token costs and improving focus), and they represent a curated, structured summary of the key concepts, making them a better basis for generating pedagogically relevant questions.

The prompt instructs the Llama 3.3 70B model to generate approximately 3–5 MCQs in a specific JSON format. The response is parsed and validated before being stored in the quizzes table with quiz_type set to 'video_quiz'. A crucial implementation detail is the handling of JSON parsing failures: if the model returns malformed JSON (which can occasionally occur), the handler catches the error, logs it, and returns a 500 response rather than allowing corrupted data to reach the database.

14.4 /api/generate-master-quiz Implementation

The revision quiz generation route is the most algorithmically interesting of the three. Upon receiving a session_id, it executes three sequential database queries: first, it fetches all videos in the session to collect their notes; second, it fetches all quiz_attempts associated with the session's quizzes; third, it extracts and deduplicates the weak_topics arrays from all attempts.

The aggregated set of unique weak topics is passed to the LLM along with all session notes as context. The prompt instructs the model to focus revision questions on the identified weak topics while using session notes as supporting context. The result is stored as a quiz_type='revision_master' record in the quizzes table.

Chapter 15 – AI Integration and Prompt Engineering

15.1 Model Selection: Llama 3.3 70B Versatile via Groq

The choice of the Llama 3.3 70B Versatile model via the Groq inference API is the result of a deliberate evaluation along three axes: capability, cost, and latency. At 70 billion parameters, the model is capable enough to produce coherent, pedagogically appropriate notes and MCQs from long educational transcripts; a task that requires semantic understanding, summarisation, and question generation capabilities simultaneously. The Groq API's use of dedicated custom hardware (Language Processing Units, LPUs) enables inference speeds that are substantially faster than traditional GPU-based inference, resulting in sub-10-second response times even for large context payloads. The free tier's generous token allowance makes this feasible for a prototype.

15.2 Prompt Engineering Strategy

All three AI-driven workflows in VibeLearn rely on carefully engineered prompts. The following principles govern the prompt design:

- Role specification: Each prompt begins by instructing the model to adopt a specific role (e.g., 'You are an expert educator and note-taker') to condition the model's output style and depth.
- Explicit format requirements: The prompt specifies the exact output format. For notes, this is structured Markdown with headings, sub-headings, and bullet points. For quizzes, this is a JSON array of question objects with a precisely defined schema.
- Schema definition: Quiz prompts include an explicit JSON schema definition to minimise parsing failures.
- Quantity specification: The number of required questions (approximately 3–5 for video quizzes, 20 for revision quizzes) is explicitly stated.
- Quality instructions: Prompts include directives such as 'ensure distractors are plausible but clearly incorrect' and 'explanations should reference the specific concept from the notes'.
- Context injection: The full relevant context (transcript for notes, notes for quizzes, notes plus weak topics for revision) is appended after the instruction block.

Prompt	Context Injected	Output Format	Key Instructions
Note Generation	Full video transcript	Structured Markdown	Topic headings, bullet points, definitions, key takeaways
Video Quiz	AI-generated notes	JSON array of MCQs	Approximately 3–5 questions, 4 options each, correct index, explanation
Revision Quiz	All session notes + weak topic labels	JSON array of MCQs	Focus on weak topics, 20 questions, same MCQ schema

Table 7: AI Prompt Design Summary

15.3 Handling Model Output Variability

A practical challenge with LLM-generated output is variability: even with detailed format instructions, models occasionally produce responses that deviate from the specified schema. VibeLearn addresses this in two ways. First, the Groq API's support for structured output mode (JSON mode) is enabled where applicable, which constrains the model's sampling process to produce valid JSON. Second, server-side parsing is wrapped in try-catch blocks, and validation checks confirm that the parsed object has the expected number of questions and that each question has the required fields before database insertion. Malformed responses cause a 500 error with an appropriate message, prompting the user to retry.

15.4 Token Management

A significant engineering consideration is token consumption. YouTube educational videos can generate transcripts of 10,000 or more words. The Llama 3.3 70B Versatile model has a context window of 128,000 tokens, which is sufficient for all realistic video lengths encountered in educational contexts. However, longer transcripts increase inference time and cost. A future optimisation (noted in Future Scope) is to implement a chunk-and-summarise strategy for transcripts exceeding a configurable word count threshold, generating notes from each chunk and then synthesising a final notes document from the chunk summaries.

Chapter 16 – Quiz Generation Engine

16.1 MCQ Data Schema

Each quiz question is stored as a JSON object within the JSONB questions array. The schema for each question object is:

```
{
  "question": "string",
  "options": ["string", "string", "string", "string"],
  "correctIndex": 0 | 1 | 2 | 3,
  "explanation": "string",
  "topic": "string"  // identifies the weak topic if answered incorrectly
}
```

The topic field is crucial for the adaptive revision system. When a user answers a question incorrectly, the topic string from that question is appended to the weak_topics array stored in the quiz_attempt record. This field must therefore be populated by the LLM with a concise, meaningful topic label (e.g., 'TCP Three-Way Handshake', 'K-Means Clustering Convergence') rather than a generic phrase.

16.2 Quiz Rendering

The quiz interface renders questions sequentially. The user selects one of four radio-button options for each question. Upon submission of the entire quiz, the interface enters an evaluation state: correct answers are highlighted in green, incorrect answers are highlighted in red with the correct answer also highlighted, and the explanation for each question is revealed. The score is calculated as the sum of correct answers, and the quiz_attempt record (including the score and the weak_topics array) is written to the database.

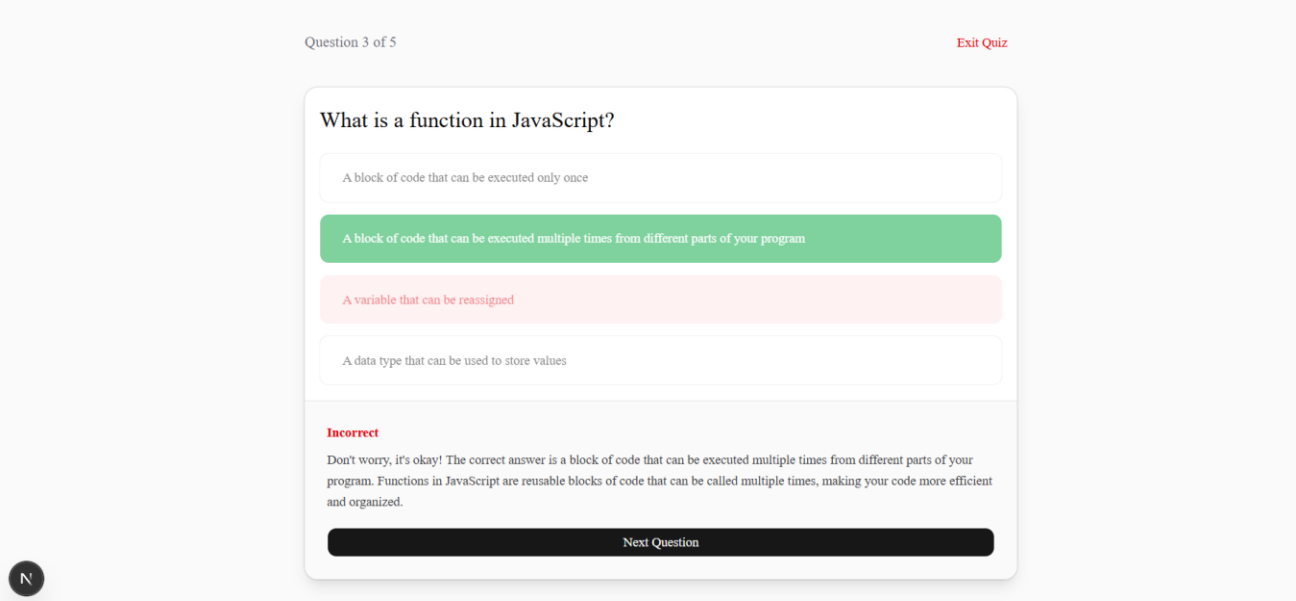

Figure 5: Quiz Interface – MCQ Engine with Instant Evaluation

Chapter 17 – Performance Tracking and Adaptive Revision Engine

17.1 Weak Topic Detection Algorithm

Weak topic detection is implemented as a straightforward aggregation process in application code. The algorithm operates as follows:

4. All quiz_attempt records associated with the current session are retrieved from Supabase.
5. Merge all weak topic labels into a single collection.
6. Remove duplicates using a JavaScript Set.
7. Pass the resulting unique weak topic list into the revision quiz prompt.
8. Generate a revision quiz using session notes and the identified weak topics.
9.

The simplicity of this approach is intentional. More sophisticated methods, such as Bayesian Knowledge Tracing or Item Response Theory (IRT), require significantly more data per learner and introduce implementation complexity disproportionate to the prototype stage. The Set-based deduplication approach is transparent, easy to reason about, and sufficient for a Phase-I MVP.

17.2 Revision Quiz Generation

The revision quiz prompt combines two contextual elements: the deduplicated weak topic list and the concatenated notes from all videos in the session. The model is instructed to focus revision questions on the identified weak topics while using session notes as supporting context.

17.3 Continuous Improvement Loop

Each revision quiz attempt generates a new quiz_attempt record. Future revision quizzes incorporate weak topics accumulated across previous attempts, allowing the revision process to continue adapting to areas where mistakes have been recorded.

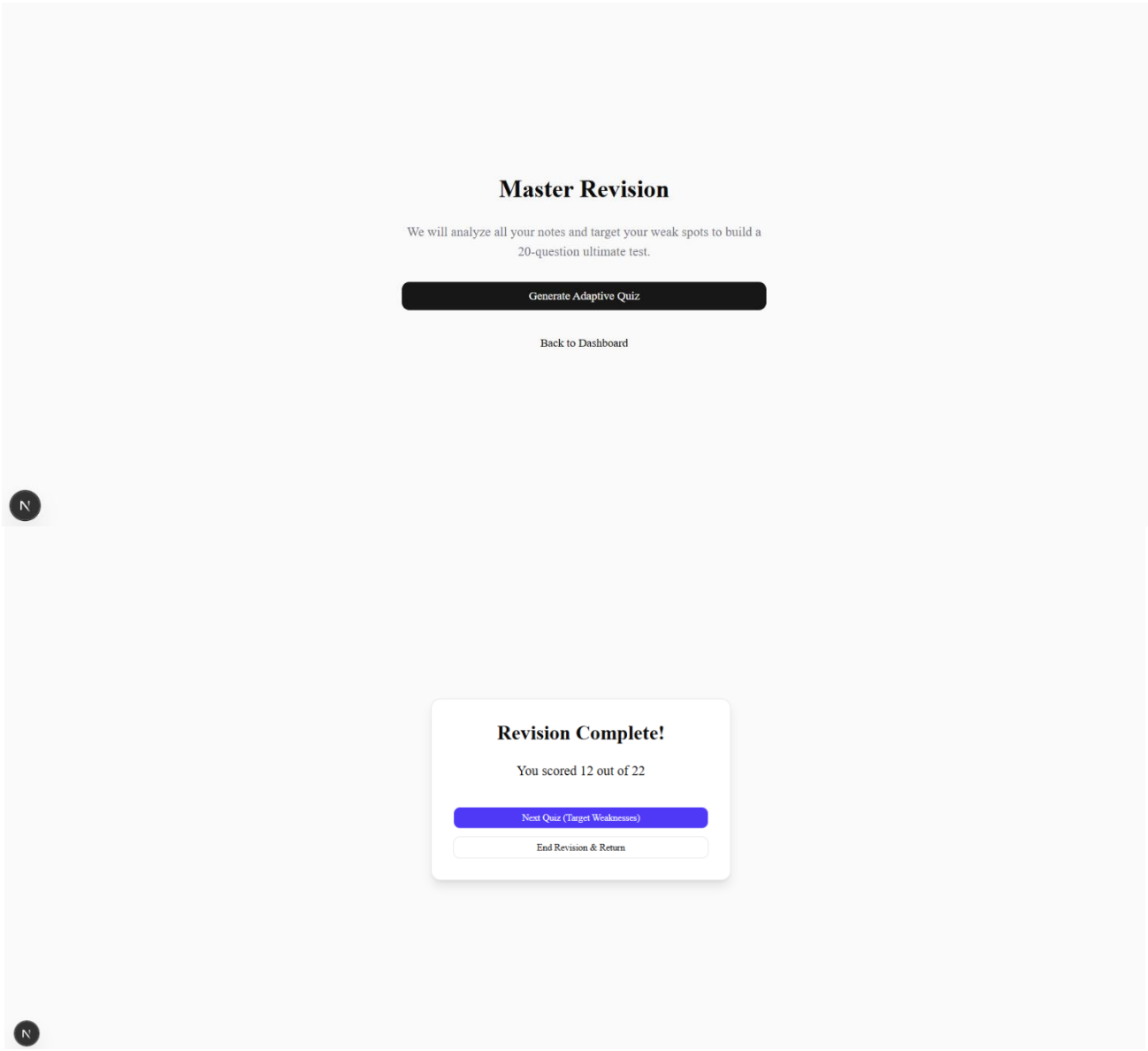

Figure 6: Revision Interface – Adaptive Revision Quiz

Chapter 18 – Authentication Design and Security Considerations

18.1 Custom Frictionless Authentication

VibeLearn intentionally does not use any standard authentication library (Clerk, Auth.js, NextAuth, Supabase Auth). This decision is driven by the prototype nature of the application and the desire to eliminate onboarding friction. The goal is for a user to begin using the platform within ten seconds of arriving at the home page, with no email confirmation, password creation, or OAuth redirect flow required.

The authentication mechanism functions as follows: the user provides a username on the home page. The server checks the users table for an existing record matching that username. If found, the user is identified by that record. If not found, a new record is created. In both cases, the user's UUID and username are stored in localStorage and loaded into React Context on subsequent page loads. There is no token, no expiry, and no cryptographic verification of identity.

18.2 Security Implications and Mitigations

The custom authentication approach has significant security implications that are fully acknowledged:

- Identity spoofing: Any user who knows another user's username can access that user's data by typing the username on the login screen. There is no password or token to prevent this. In a production context, this is unacceptable. In a prototype context with no sensitive personal data and the expectation of trusted users, this is an acceptable tradeoff.
- localStorage persistence: Storing the user UUID in localStorage exposes it to JavaScript running in the same origin. This is acceptable in the absence of third-party scripts, which are not present in the current application.
- API key security: All API keys (Groq, Supabase service role) are stored as server-side environment variables prefixed with GROQ_API_KEY and SUPABASE_SERVICE_KEY respectively. These are never passed to the client bundle and are inaccessible from the browser. This is the primary security control in the application.
- Input validation: All API route inputs are validated for presence and basic format before use. YouTube URL parsing uses a well-tested regular expression to prevent injection of arbitrary strings as video IDs.

18.3 Future Authentication Roadmap

Phase-II authentication will replace the custom mechanism with a proper authentication system. The current leading candidate is Supabase Auth with GitHub OAuth, which requires minimal additional UI work (a single 'Sign in with GitHub' button), provides cryptographically secure session tokens stored in HttpOnly cookies, and eliminates all identity spoofing vulnerabilities.

Chapter 19 – Challenges Faced and Engineering Tradeoffs

19.1 Challenges Encountered

19.1.1 Transcript Quality Variability

YouTube's auto-generated captions vary significantly in quality. For professionally produced educational content (university lectures, coding tutorials from major channels), captions are generally accurate. For content with heavy technical jargon, accented speech, or poor audio quality, captions may contain errors that propagate into the generated notes. The current implementation has no mechanism to detect or correct caption errors. A future mitigation is to expose a confidence indicator or allow users to manually edit notes post-generation.

19.1.2 LLM JSON Output Consistency

Despite using Groq's JSON mode, the model occasionally generates JSON that is syntactically valid but semantically incorrect; for example, a correctIndex value outside the range 0–3, or a missing topic field. Robust validation of the parsed JSON object against a Zod schema was implemented partway through development after encountering this issue in testing. This added approximately 50 lines of validation code but significantly improved reliability.

19.1.3 Transcript Retrieval Failures

The youtube-transcript package depends on YouTube's internal caption delivery endpoints, which are not part of any documented public API. These endpoints occasionally rate-limit requests or return errors for videos where captions are available but temporarily inaccessible. Exponential back-off retry logic was considered but not implemented in Phase-I due to time constraints. Users are instead shown an error message advising them to retry after a brief delay.

19.1.4 Next.js App Router Learning Curve

The Next.js App Router, introduced as stable in version 13.4, represents a significant architectural departure from the Pages Router. The distinction between Server Components and Client Components, the placement of 'use client' directives, and the handling of async Server Component data fetching required careful study and several refactoring cycles before a clean, correct architecture was achieved. In particular, understanding that useState and event handlers require Client Components while data fetching prefers Server Components was a non-trivial conceptual shift.

19.2 Engineering Tradeoffs Analysis

Decision	Chosen Approach	Advantage	Disadvantage
Authentication	Custom localStorage	Zero friction onboarding	No identity verification
Architecture	Monolithic Next.js	Simple deployment, no CORS	Not microservice-scalable
State Management	React Context + useState	No external library dependency	Not suitable for complex state
AI Model	Llama 3.3 70B / Groq	Free tier, high speed, capable	External API dependency
Quiz Context	Notes (not transcript)	Shorter context, focused output	May miss transcript nuances
JSONB vs normalised	JSONB for questions/topics	Flexible schema, simpler queries	Harder to query sub-fields
Weak topic algo	Frequency counting	Simple, interpretable, fast	Not probabilistically rigorous
Row Level Security	Disabled (Phase-I)	Faster development	All data globally accessible

Table 9: Engineering Tradeoffs Analysis

Chapter 20 – Testing Strategy and Test Cases

20.1 Testing Approach

Given the prototype nature of this project and the sole-developer context, the testing strategy is primarily manual and exploratory, supplemented by systematic test case execution for critical paths. Automated unit testing (Jest, Vitest) was considered but deferred to Phase-II to avoid adding tooling overhead during the initial MVP sprint. The testing strategy covers four layers: input validation, API route behaviour, AI output quality, and end-to-end workflow correctness.

20.2 Test Cases

TC	Description	Input	Expected Output	Result
TC-01	New user registration	Unique username submitted	User record created, redirected to dashboard	Pass
TC-02	Returning user login	Existing username submitted	Existing user retrieved, redirected to dashboard	Pass
TC-03	Create new session	Session title submitted	Session row created, workspace opened	Pass
TC-04	Valid YouTube URL (long form)	https://youtube.com/watch?v=abc123	Notes generated and displayed	Pass
TC-05	Valid YouTube URL (short form)	https://youtu.be/abc123	Notes generated and displayed	Pass
TC-06	Invalid URL submitted	https://vimeo.com/123	Error message: Invalid YouTube URL	Pass
TC-07	Video without captions	YouTube URL of caption-disabled video	Error message: Transcript not available	Pass
TC-08	Quiz generation	Valid video_id with notes	10 MCQs generated and rendered	Pass
TC-09	Quiz submission	All 10 answers selected	Score displayed, weak topics identified	Pass
TC-10	Revision quiz generation	Session with 2+ quiz attempts	Adaptive quiz targeting weak topics generated	Pass
TC-11	End session	End Session button clicked	Session status updated to 'completed'	Pass
TC-12	Access completed session	Completed session selected from dashboard	Notes and quiz history accessible (read-only)	Pass

Table 8: Test Cases – Core Workflows

Chapter 21 – Results and Discussion

21.1 Feature Completeness

All eight functional modules defined in the project objectives have been implemented and verified through the test cases documented in Chapter 20. The platform successfully completes the full learning loop; from YouTube URL input to adaptive revision quiz generation; without manual intervention beyond user interaction with the interface. The Phase-I MVP is functionally complete relative to its defined scope.

21.2 AI Output Quality Observations

Over approximately 40 test runs during development, the following qualitative observations about AI output quality were recorded:

- Note generation quality: For well-captioned educational videos (MIT OpenCourseWare lectures, CS50, FreeCodeCamp tutorials), the generated notes are accurate, well-structured, and comparable in quality to notes a diligent student might produce manually. For lower-quality transcripts, the notes contain some errors inherited from caption inaccuracies but remain generally useful.
- MCQ quality: Generated questions range from straightforward recall questions to analytical application questions. Distractors are generally plausible, and explanations are accurate and informative. Occasional questions are too easy (single-word factual recall) or too similar to each other.
- Topic label quality: The topic field generated by the LLM is reasonably descriptive in most cases, enabling meaningful weak topic identification. Occasional generic labels ('General Concepts') reduce the specificity of the adaptive revision.

21.3 Performance Metrics

Informal testing indicated that note generation, quiz generation, revision quiz generation, and dashboard loading performed at acceptable speeds for prototype usage. No formal benchmarking or controlled performance evaluation was conducted during Phase-I.

Chapter 22 – Limitations

22.1 Known Limitations

- Authentication security: The custom localStorage-based authentication provides no identity verification. This is the single most significant limitation relative to a production-ready system.
- Transcript dependency: The platform is entirely dependent on the availability of YouTube captions. Videos without captions; estimated to be a small but non-trivial fraction of educational content; cannot be processed.
- Language restriction: Only English-language transcripts are currently supported. The Llama 3.3 70B model is capable of multilingual processing, but the prompts are English-only.
- Single-user session isolation: There is no row-level security in the database. Any user who knows another user's UUID can query their session data directly through the Supabase client. This is mitigated by the fact that UUIDs are not exposed in the UI, but it is not a genuine security control.
- No offline functionality: The platform requires active internet connectivity for all operations. There is no caching of generated content for offline access.
- Adaptive algorithm simplicity: The current implementation aggregates and deduplicates weak topics without weighting, ranking, or statistical analysis.
- No error recovery for partial failures: If a database write fails after a successful Groq API call, the generated content is lost. There is no retry or recovery mechanism.

Chapter 23 – Future Scope

23.1 Phase-II Development Priorities

Feature	Description	Priority
Supabase Auth + GitHub OAuth	Replace custom auth with cryptographically secure sessions	Critical
Row Level Security	Enable per-user data isolation at the database layer	Critical
Spaced Repetition Scheduler	Implement SM-2 or FSRS algorithm for optimal review scheduling	High
Transcript Chunking	Handle very long transcripts by chunk-and-summarise strategy	High
Zod Schema Validation	Full runtime validation of all LLM outputs against Zod schemas	High
Multi-language Support	Process transcripts in languages other than English	Medium
Flashcard Generation	Generate Anki-compatible flashcard decks from session notes	Medium
User Analytics Dashboard	Visual display of performance trends over time	Medium
Collaborative Sessions	Allow multiple users to contribute videos to a shared session	Low
Mobile Application	React Native or Progressive Web App for mobile learning	Low

Table 10: Future Scope Prioritisation

Chapter 24 – Conclusion

VibeLearn represents a functionally complete Phase-I MVP of an AI-powered personalised learning and adaptive revision platform. The project demonstrates that modern full-stack tooling; specifically Next.js, Supabase, and the Groq API; enables a single developer to build a sophisticated, multi-feature application within a focused development sprint. The platform successfully addresses all three of the core problems identified in the Problem Statement: it generates structured notes automatically, provides MCQ-based formative assessment, and implements an adaptive revision mechanism that generates quizzes based on previously identified weak topics.

From a technical perspective, the project delivered practical, hands-on experience with several technologies of direct relevance to the author's professional objectives in software development: the Next.js App Router architecture, TypeScript-first development, PostgreSQL schema design, REST API design within a monolith, and prompt engineering for structured LLM output generation. Each of these skills was acquired not through tutorial exercises but through the friction of real problem-solving, which produces significantly deeper and more durable understanding.

The engineering decisions made throughout the project; from the choice of localStorage-based authentication to the use of JSONB for quiz data; were made consciously, with explicit recognition of the tradeoffs involved. This reflective engineering practice, documented throughout this report, is itself a valuable output of the project.

The limitations of the Phase-I MVP are well-understood and have been used to define a clear Phase-II roadmap. The most critical future priority is the replacement of the custom authentication system with a cryptographically secure alternative, followed by the implementation of database row-level security and a spaced repetition scheduler to complete the adaptive learning feature set.

VibeLearn began as a personal educational problem and evolved into a demonstration of the author's capacity to design, implement, and critically evaluate a full-stack software system. It stands as the centrepiece of the author's technical portfolio for the upcoming Software Development Engineer internship season.

References

[1] Ebbinghaus, H. (1885). Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie. Duncker & Humblot.

[2] Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. arXiv:2005.14165.

[3] Elkins, S., et al. (2023). How Teachers Can Use Large Language Models and Bloom's Taxonomy to Create Educational Quizzes. arXiv:2301.05914.

[4] Bunt, A., Lount, M., & Lauzon, C. (2014). Are Explanations Always Important? A Study of Deployed, Low-Cost Intelligent Tutoring Systems. ITS 2014.

[5] Sleeman, D., & Brown, J. S. (1982). Intelligent Tutoring Systems. Academic Press, London.

[6] Next.js Documentation. (2024). App Router. https://nextjs.org/docs/app

[7] Supabase Documentation. (2024). JavaScript Client Library. https://supabase.com/docs/reference/javascript

[8] Groq Documentation. (2024). API Reference. https://console.groq.com/docs

[9] Meta AI. (2024). Llama 3.3 Model Card. https://llama.meta.com

[10] youtube-transcript npm package. https://www.npmjs.com/package/youtube-transcript

[11] Shadcn/UI Documentation. (2024). https://ui.shadcn.com

[12] Tailwind CSS Documentation. (2024). https://tailwindcss.com/docs

[13] PostgreSQL JSONB Documentation. (2024). https://www.postgresql.org/docs/current/datatype-json.html

Appendix

Appendix A: Project Statistics

Metric	Value
Total Pages	5
API Routes	3
Database Tables	5
Approximate LOC	~1,500
Functional Modules	8
AI-Powered Features	4
Major Technologies	10
Development Duration	~4 weeks (Phase-I)
Language	TypeScript
Runtime	Node.js (Next.js)

Table A1: Project Statistics Summary

Appendix B: Environment Variables

```
# .env.local
GROQ_API_KEY=<groq_api_key>
NEXT_PUBLIC_SUPABASE_URL=<supabase_project_url>
NEXT_PUBLIC_SUPABASE_ANON_KEY=<supabase_anon_key>
SUPABASE_SERVICE_ROLE_KEY=<supabase_service_role_key>
```

Appendix C: Directory Structure

```
vibelearn/
├── app/
│   ├── api/
│   │   ├── generate-notes/route.ts
│   │   ├── generate-quiz/route.ts
│   │   └── generate-master-quiz/route.ts
│   ├── dashboard/page.tsx
│   ├── session/[id]/page.tsx
│   ├── quiz/[id]/page.tsx
│   ├── revision/[id]/page.tsx
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── VideoCard.tsx
│   ├── QuizQuestion.tsx
│   ├── ScoreDisplay.tsx
│   ├── SessionCard.tsx
│   └── MarkdownRenderer.tsx
├── context/
│   └── AuthContext.tsx
├── lib/
│   ├── supabase.ts
│   └── groq.ts
├── types/
│   └── index.ts
├── .env.local
├── next.config.ts
├── tailwind.config.ts
└── package.json
```